

I040-2-8461

Open Source Software

07. Linux

Dr. Kyudong Park

School of Information Convergence



Contents

- [리눅스 기초](#)
- Directory and File
- 문서 편집
- Shell (셸)
- 접근 권한
- 프로세스 관리
- 리눅스 git 세팅

리눅스 기초

- 리눅스는 유닉스와의 호환성 및 공개 소프트웨어라는 점에 큰 주목을 받음
- 임베디드 시스템, 모바일 기기 등 다양한 환경에서 사용
- Linus Torvalds (1991)

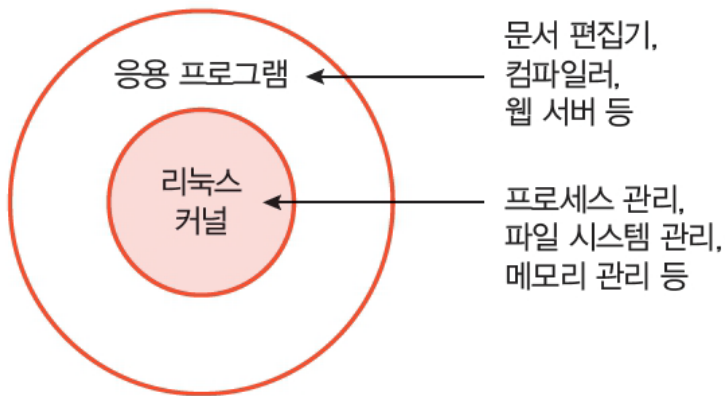
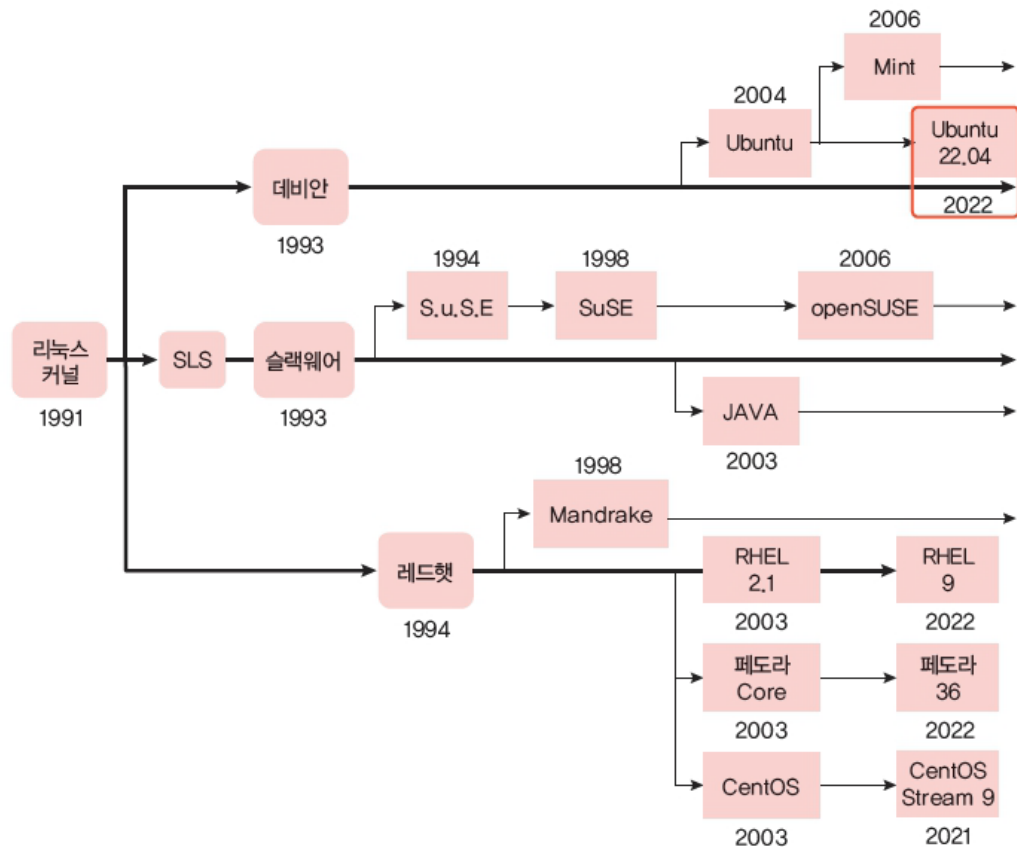


그림 1-1 커널과 응용 프로그램

리눅스 기초

- 리눅스 배포판

- 리눅스 커널과 응용 프로그램을 적절하게 구성한 버전
- 누구나 본인이 원하는대로 구성하여 배포 가능



리눅스 기초

- ubuntu 의 버전 관리

- 배포판이 나온 연도와 월로 구성(예: 22.04 – 2022년 4월 버전)
- 버전에 따라 코드명 부여(예: 21.10 – Impish Indri, 장난꾸러기 여우원숭이)
- 일반 버전과 장기 지원 버전
 - 장기 지원 버전은 4번에 한 번씩 발표하고 5년간 지원
 - 일반 버전은 4월, 10월에 발표하고 9개월간 지원

20.04 (LTS)	Focal Fossa (중심의 포사)	5.4.0	2020년 04월 23일	~ 2025년 04월
20.10	Groovy Gorilla (멋진 고릴라)	5.8.0	2020년 10월 22일	~ 2021년 07월
21.04	Hirsute Hippo (털이 많은 하마)	5.11.0	2021년 04월 22일	~ 2022년 01월
21.10	Impish Indri (장난꾸러기 여우원숭이)	5.13.0	2021년 10월14일	~ 2022년 07월

리눅스 기초

- 리눅스의 특징과 구조
 - 리눅스는 공개 소프트웨어이며 무료로 사용할 수 있다.
 - 유닉스와 완벽한 호환성을 유지한다.
 - 서버용 운영체제로 많이 사용된다.
 - 편리한 GUI 환경을 제공한다.

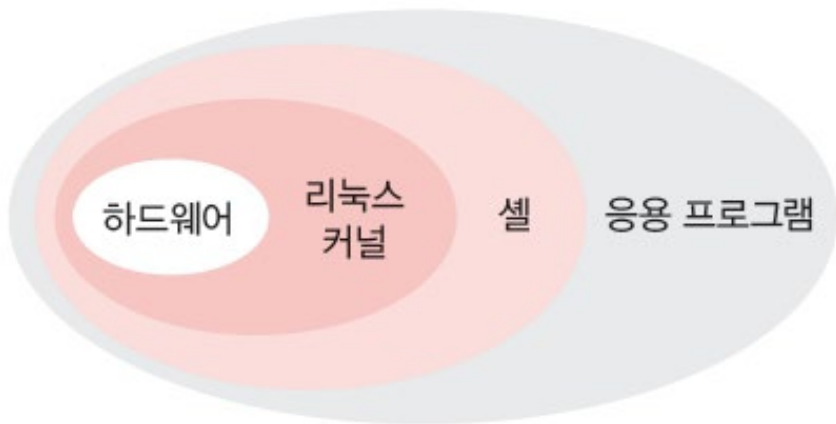


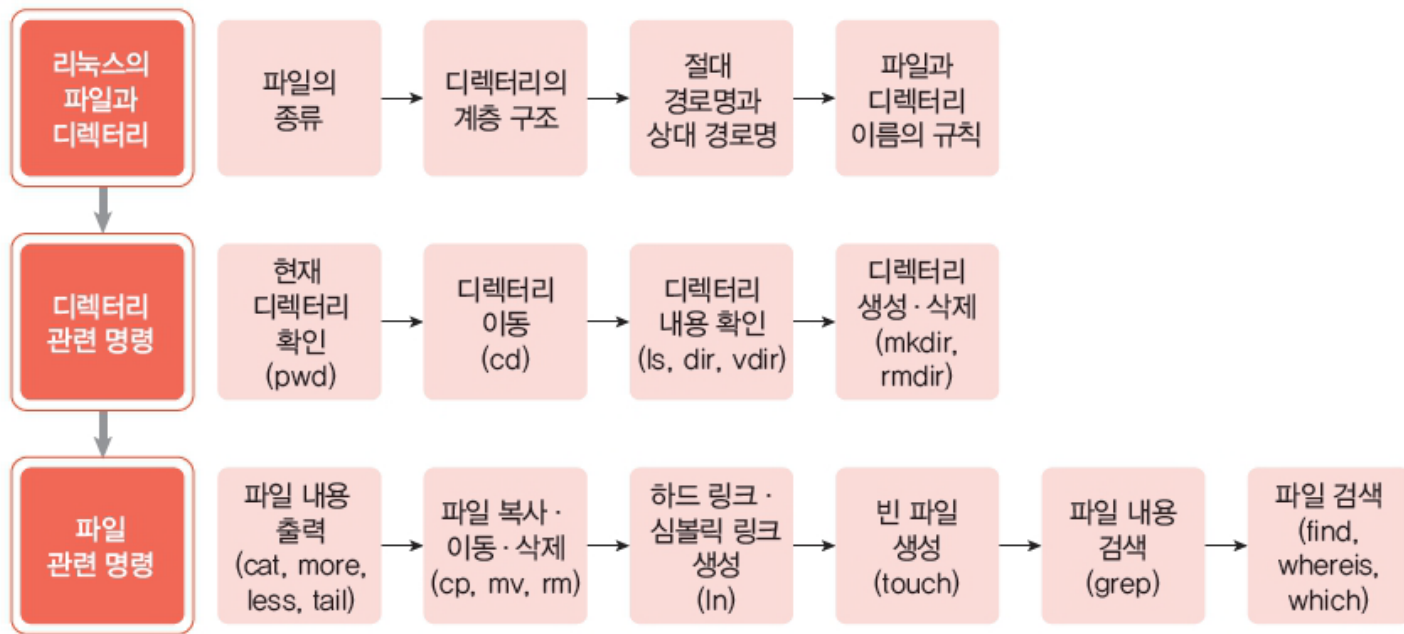
그림 1-5 리눅스의 구조

Contents

- 리눅스 기초
- [Directory and File](#)
- 문서 편집
- Shell (셸)
- 접근 권한
- 프로세스 관리
- 리눅스 git 세팅

Directory & File

- Directory = 폴더



Directory & File

- File: 관련있는 정보들의 집합
- File의 종류
 - 일반 파일: 텍스트 파일, 실행파일 이미지 파일 등 주로 데이터를 저장
 - Directory
 - 리눅스에서는 디렉터리도 파일로 취급
 - 해당 디렉터리에 저장된 파일이나 하위 디렉터리에 대한 정보 저장
 - Symbolic link
 - 원본 파일을 다른 파일명으로 지정한것
 - Windows의 바로가기 파일과 유사
 - Device file
 - 하드디스크나 키보드 같은 각종 장치도 파일로 취급

Directory & File

- 파일의 종류를 알아보는 명령 : file

```
user1@myubuntu:~$ file .profile
.profile: ASCII text
user1@myubuntu:~$ file 다운로드
다운로드: directory
```



텍스트 파일, 디렉터리

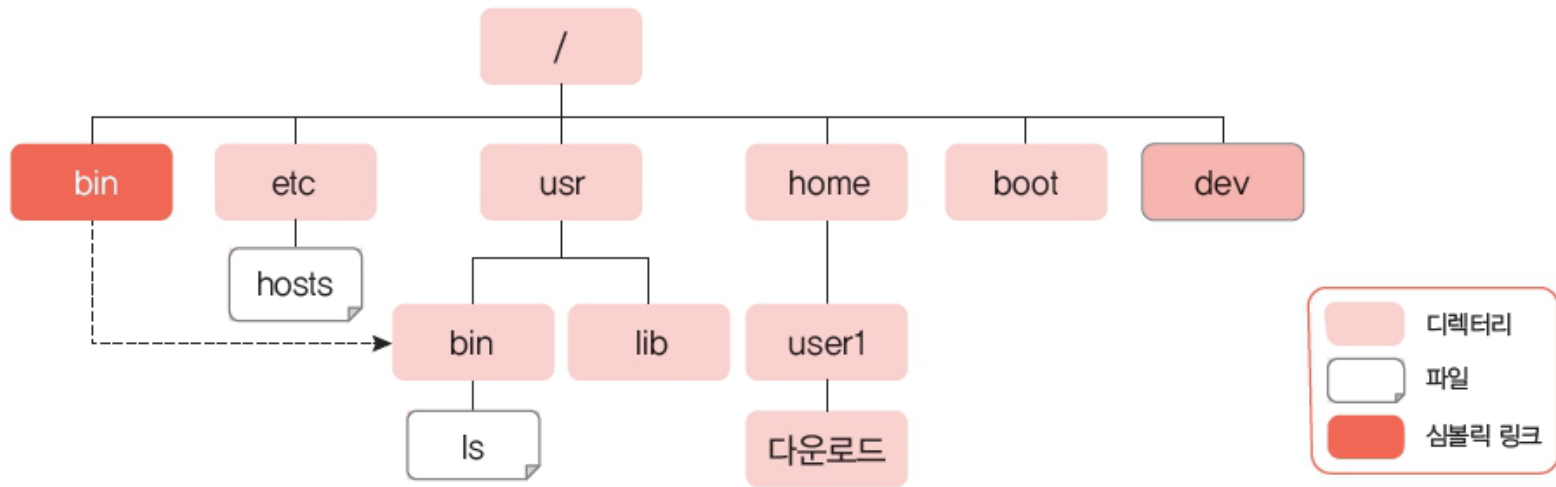
```
user1@myubuntu:~$ file /bin/bash
/bin/bash: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
(생략)
```



실행 파일

Directory & File

- Directory의 계층구조
 - root directory 의 개념
 - parent, child directory



Directory & File

- Directory 의 주요 기능

디렉터리	기능
dev	장치 파일이 담긴 디렉터리다.
home	사용자 홈 디렉터리가 생성되는 디렉터리다.
media	DVD/CD나 USB 같은 외부 장치를 연결(마운트라고 함)하는 디렉터리다.
opt	추가 패키지가 설치되는 디렉터리다.
root	root 계정의 홈 디렉터리다. 루트(/) 디렉터리와 다른 것이므로 혼동하지 않도록 한다.
sys	리눅스 커널과 관련된 파일이 있는 디렉터리다.
usr	기본 실행 파일과 라이브러리 파일, 헤더 파일 등 많은 파일이 있다. 참고로 usr는 'Unix System Resource'의 약자다.
boot	부팅에 필요한 커널 파일을 가지고 있다.
etc	리눅스 설정을 위한 각종 파일을 가지고 있다.
lost+found	파일 시스템에 문제가 발생하여 복구할 경우, 문제가 되는 파일이 저장되는 디렉터리로 보통은 비어 있다.
mnt	파일 시스템을 임시로 마운트하는 디렉터리다.
proc	프로세스 정보 등 커널 관련 정보가 저장되는 디렉터리다.
run	실행 중인 서비스와 관련된 파일이 저장된다.
srv	FTP나 Web 등 시스템에서 제공하는 서비스의 데이터가 저장된다.
tmp	시스템 사용 중에 발생하는 임시 데이터가 저장된다. 이 디렉터리에 있는 파일은 재시작하면 모두 삭제된다.
var	시스템 운영 중에 발생하는 데이터나 로그 등 내용이 자주 바뀌는 파일이 주로 저장된다.

Directory & File

- 절대경로명, 상대경로명

구분	특징
절대 경로명	• 반드시 /로 시작한다. • 루트 디렉터리부터 시작하여 특정 파일이나 디렉터리의 위치에 이르기까지 중간에 있는 모든 디렉터리의 이름을 표시한다. • 특정 위치를 가리키는 절대 경로명은 항상 동일하다.
상대 경로명	• / 이외의 문자로 시작한다. • 현재 디렉터를 기준으로 서브 디렉터리로 내려가면 그냥 서브 디렉터리명으로 시작한다. • 현재 디렉터를 기준으로 상위 디렉터리로 가려면 ..(마침표 두 개)로 시작한다. • 상대 경로명은 현재 디렉터리가 어디냐에 따라 달라진다.

Directory & File

- Directory & File 명명 규칙

규칙	• 파일과 디렉터리의 이름에는 /를 사용할 수 없다. /는 경로명에서 구분자로 사용하기 때문이다. • 파일과 디렉터리의 이름에는 알파벳, 숫자, 하이픈(-), 언더스코어(_), 마침표(.)를 사용할 수 있다. • 파일과 디렉터리의 이름에는 /를 제외한 특수문자를 사용할 수 있으나 >, , :, &와 공백문자의 사용은 피하는 것이 좋다. • 파일과 디렉터리 이름의 영문은 대문자와 소문자를 구별하여 다른 글자로 취급한다. • 파일과 디렉터리의 이름이 .(마침표)로 시작하면 숨김 파일로 간주한다. • 파일 이름에는 널 문자("\0")를 사용하지 않는다.
예	• 좋은 이름 예: game.txt, hello.c, test, sample1 1 • 나쁜 이름 예: &game, *dir, my home, game\ • 사용할 수 없는 이름 예: myhome/, /test, bad/name

Directory & File

- 주요 명령어

pwd

- **기능** 현재 디렉터리의 위치를 확인한다. 즉 현재 디렉터리의 절대 경로를 출력한다.
- **형식** pwd

```
user1@myubuntu:~$ pwd  
/home/user1
```

Directory & File

- 주요 명령어

cd

- **기능** 지정한 디렉터리로 이동한다.
- **형식** cd [디렉터리]
- **사용 예** cd, cd /tmp, cd 다운로드

```
user1@myubuntu:~$ cd /tmp
user1@myubuntu:/tmp$ pwd
/tmp
```

```
user1@myubuntu:/tmp$ cd ../usr/lib
user1@myubuntu:/usr/lib$ pwd
/usr/lib
```


Directory & File

- 주요 명령어

ls

- 기능** 디렉터리의 내용을 출력한다.
- 형식** `ls [옵션] [디렉터리(파일)]`
- 옵션**
 - `-a`: 숨김 파일을 포함하여 모든 파일의 목록을 출력한다.
 - `-d`: 디렉터리 자체의 정보를 출력한다.
 - `-i`: 첫 번째 행에 inode 번호를 출력한다.
 - `-l`: 파일의 상세 정보를 출력한다.
 - `-A`: `.`(마침표)와 `..`(마침표 두 개)를 제외한 모든 파일 목록을 출력한다.
 - `-F`: 파일의 종류를 표시한다(*: 실행 파일, /: 디렉터리, @: 심볼릭 링크).
 - `-L`: 심볼릭 링크 파일의 경우 원본 파일의 정보를 출력한다.
 - `-R`: 하위 디렉터리의 목록까지 출력한다.
- 사용 예** `ls` `ls -F` `ls -al /tmp`

Directory & File

- 현재 디렉터리 내용 확인: `ls`

```
user1@myubuntu:~$ ls
```

```
공개 다운로드 문서 바탕화면 비디오 사진 음악 템플릿
```

- 숨김 파일 확인하기: `ls -a`
 - 숨김파일: `.` 으로 시작하는 파일

```
user1@myubuntu:~$ ls -a
```

```
.          .bashrc  .profile          문서      음악
..         .cache   .sudo_as_admin_successful  바탕화면  템플릿
.bash_history .config  공개              비디오
.bash_logout .local   다운로드          사진
```

- 파일의 종류 표시: `ls -F`

```
user1@myubuntu:~$ ls -F
```

```
공개/  다운로드/  문서/  바탕화면/  비디오/  사진/  음악/  템플릿/
```

Directory & File

- 상세 정보 출력하기 : ls -l

```
user1@myubuntu:~$ ls -l
```

합계 32

```
drwxr-xr-x 2 user1 user1 4096 11월 21 12:05 공개
drwxr-xr-x 2 user1 user1 4096 11월 21 12:05 다운로드
drwxr-xr-x 2 user1 user1 4096 11월 21 12:05 문서
drwxr-xr-x 2 user1 user1 4096 11월 21 12:05 바탕화면
drwxr-xr-x 2 user1 user1 4096 11월 21 12:05 비디오
drwxr-xr-x 2 user1 user1 4096 11월 21 12:05 사진
drwxr-xr-x 2 user1 user1 4096 11월 21 12:05 음악
drwxr-xr-x 2 user1 user1 4096 11월 21 12:05 템플릿
```

필드 번호	필드 값	의미
1	d	다음과 같은 파일 종류를 나타낸다. -: 일반(정규) 파일 d: 디렉터리 파일 l: 심볼릭 링크 파일 b: 블록 단위로 읽고 쓰는 블록 장치 파일 c: 섹터 단위로 읽고 쓰는 문자 장치 파일 p: 파이프 파일(프로세스 간 통신에 사용되는 특수 파일) s: 소켓(네트워크 통신에 사용되는 특수 파일)
2	rw-r--r--	파일 접근 권한, 파일의 소유자, 그룹, 기타 사용자가 읽고 수정하고 실행할 수 있는 권한이 어떻게 부여되어 있는지를 보여준다.
3	2	하드 링크의 개수
4	user1	파일 소유자
5	user1	파일이 속한 그룹
6	4096	파일 크기(바이트 단위)
7	11월 21 12:05	파일이 마지막으로 수정된 시간
8	공개	파일명

Directory & File

- 주요 명령어

mkdir

- 기능** 디렉터리를 생성한다.
- 형식** mkdir [옵션] [디렉터리]
- 옵션** -p: 하위 디렉터를 계층적으로 생성할 때 중간 단계의 디렉터리가 없으면 자동으로 중간 단계 디렉터를 생성하고 지정한 디렉터를 생성한다.
- 사용 예** mkdir temp

```
user1@myubuntu:~$ mkdir temp
user1@myubuntu:~$ ls temp
user1@myubuntu:~$ ls
temp  공개  다운로드  문서  바탕화면  비디오  사진  음악  템플릿
```

```
user1@myubuntu:~$ mkdir tmp1 tmp2 tmp3
user1@myubuntu:~$ ls
tmp1  tmp2  공개  문서  비디오  음악
tmp1  tmp3  다운로드  바탕화면  사진  템플릿
```

Directory & File

- 주요 명령어

rmmdir

- **기능** 디렉터리를 삭제한다.
- **형식** `rmmdir [옵션] [디렉터리]`
- **옵션** `-p`: 지정한 디렉터를 삭제하고, 그 디렉터리의 부모 디렉터리가 빈 디렉터리일 경우 부모 디렉터리도 자동으로 삭제한다.
- **사용 예** `rmmdir temp`

```
user1@myubuntu:~$ rmmdir temp
rmmdir: failed to remove 'temp': 디렉터리가 비어있지 않음
```

- `rm -r` 으로 실행 : 파일이 들어있는 디렉터리의 삭제

Directory & File

- 주요 명령어

cat

- 기능** 파일 내용을 출력한다.
- 형식** cat [옵션] [파일]
- 옵션** -n: 행 번호를 붙여서 출력한다.
- 사용 예** cat file1 cat -n file1

```
user1@myubuntu:~$ cat /etc/hosts
127.0.0.1 localhost
127.0.1.1 myubuntu

# The following lines are desirable for IPv6 capable hosts
::1 ip6-localhost ip6-loopback
fe00::0 ip6-localnet
ff00::0 ip6-mcastprefix
```

```
user1@myubuntu:~$ cat -n /etc/hosts
1 127.0.0.1 localhost
2 127.0.1.1 myubuntu
3
4 # The following lines are desirable for IPv6 capable hosts
5 ::1 ip6-localhost ip6-loopback
6 fe00::0 ip6-localnet
7 ff00::0 ip6-mcastprefix
8 ff02::1 ip6-allnodes
9 ff02::2 ip6-allrouters
```

Directory & File

- 주요 명령어

less

- **기능** 파일 내용을 화면 단위로 출력한다.
- **형식** less [파일]
- **사용 예** less file1

- 화면 넘기기

- ↑, ↓, ←, →, PageUp, PageDown도 사용 가능
 - / + 문자열 입력 : 문자열 검색
 - q : 종료

Directory & File

- 주요 명령어

tail

- 기능** 파일 뒷부분의 몇 행을 출력한다.
- 형식** tail [옵션] [파일]
- 옵션** +행 번호: 지정한 행부터 끝까지 출력한다.
 - 숫자: 화면에 출력할 행의 수를 지정한다(기본 값은 10).
 - f: 파일 출력을 종료하지 않고 주기적으로 계속 출력한다.

```
user1@myubuntu:~$ tail -7 /etc/services
asp          27374/udp
csync2       30865/tcp
dircproxy    57000/tcp
tfido        60177/tcp
fido         60179/tcp

# Local services
```


Directory & File

- 주요 명령어

cp

- **기능** 파일이나 디렉터리를 복사한다.
- **형식** `cp [옵션] [파일1(디렉터리1)] [파일2(디렉터리2)]`
- **옵션** `-i`: 파일2가 이미 존재하면 덮어쓸 것인지 물어본다.
`-r`: 디렉터리를 복사할 때 지정한다.
- **사용 예**
`cp file1 file2`
`cp f1 f2 f3 dir1` 인자를 여러 개 지정하면 마지막에 지정한 디렉터리로 파일들을 모두 복사
`cp -r dir1 dir2`

Directory & File

- 주요 명령어

mv

- **기능** 파일 또는 디렉터리를 이동하거나 이름을 바꾼다.
- **형식** mv [옵션] [파일1(디렉터리1)] [파일2(디렉터리2)]
- **옵션** -i: 파일2(디렉터리2)가 존재하면 덮어쓸 것인지 물어본다.
- **사용 예** mv file1 file2

- 동일 경로에서 이름만 다르게 주면 파일명이 변경됨

Directory & File

- 주요 명령어

rm

- **기능** 파일을 삭제한다.
- **형식** rm [옵션] [파일 또는 디렉터리]
- **옵션** -i: 파일을 정말 삭제할 것인지 확인한다.
-r: 디렉터리를 삭제할 때 지정한다.
- **사용 예** rm file rm -r dir

Directory & File

- 주요 명령어
 - 파일 링크 생성

ln

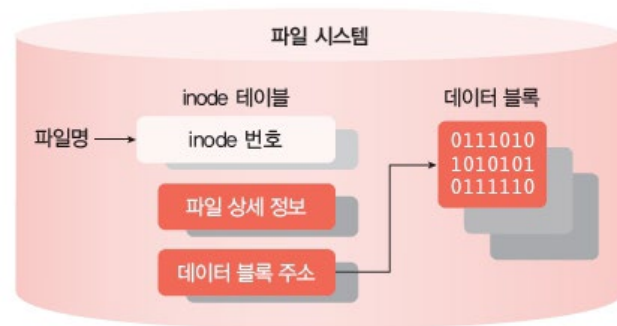
- **기능** 파일의 링크를 생성한다.
- **형식** `ln [옵션] [원본 파일] [링크 파일]`
- **옵션** `-s`: 심볼릭 링크 파일을 생성한다.
- **사용 예** `ln test hdtest      ln -s test sbtest`

- 하드링크와 심볼릭링크 생성 후 `inode` 및 `ls -li` 비교
 - 파일의 종류가 다르게 표기
 - 파일명 뒤에 원본 파일 이름 표시

Directory & File

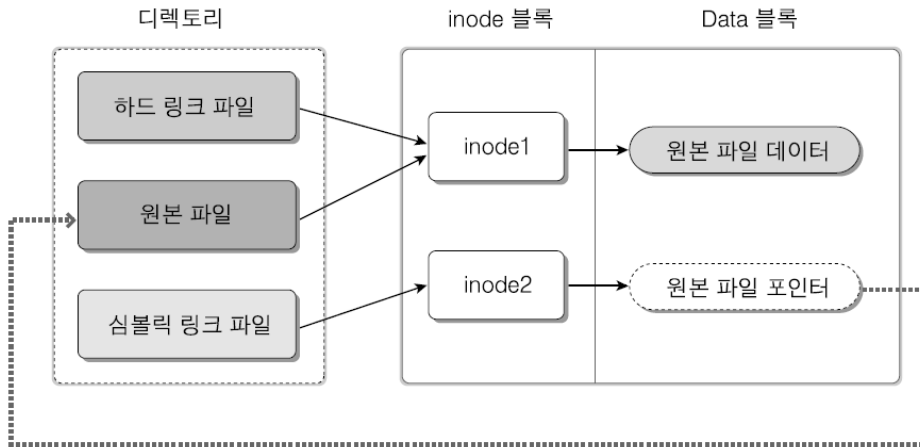
- 리눅스 파일의 구성
 - 파일명 + inode + 데이터블록
 - 파일의 inode 번호 검색 : `ls -li`

```
user1@myubuntu:~/linux_ex/ch2$ ls -li
1058724 data1  1058725 temp  1058745 test.org
```



Directory & File

- 파일 링크의 개념
 - 기존 파일에 새로운 이름을 붙이는 것
 - Hard link
 - 한 파일에 여러 개의 이름 붙이기
 - Symbolic link
 - 윈도우의 바로가기와 동일



Directory & File

- 주요 명령어
 - 파일 내용 검색하기

grep

- **기능** 지정한 패턴이 포함된 행을 찾는다.
- **형식** `grep [옵션] [패턴] [파일]`
- **옵션**
 - i: 대문자 · 소문자를 모두 검색한다.
 - l: 지정한 패턴이 포함된 파일명을 출력한다.
 - n: 행 번호를 출력한다.
- **사용 예**
 - `grep root /etc/passwd`
 - `grep -n unix ~/.txt`
 - `grep -l hello *.c`

```
user1@myubuntu:~/linux_ex/ch2$ cp /etc/services data
user1@myubuntu:~/linux_ex/ch2$ grep NNTP data
nntps          563/tcp      snttp         # NNTP over SSL
```

```
user1@myubuntu:~/linux_ex/ch2$ grep -n NNTP data
121:nntps 563/tcp snttp # NNTP over SSL
```

Directory & File

- 주요 명령어
 - 파일 찾기

find

- **기능** 지정한 위치에서 검색 조건에 맞는 파일을 찾는다.
- **형식** `find [경로] [검색 조건] [동작]`
- **검색 조건** `-name filename`: 파일명으로 검색한다.
 - `-type` 파일 종류: 파일 종류로 검색한다.
 - `-user loginID`: 지정한 사용자가 소유한 모든 파일을 검색한다.
 - `-perm` 접근 권한: 지정한 사용 권한과 일치하는 파일을 검색한다.
- **동작** `-exec 명령 {} \;`: 검색된 파일에 명령을 실행한다.
 - `-ok 명령 {} \;`: 사용자의 확인을 받아서 명령을 실행한다.
 - `-print`: 검색된 파일의 절대 경로명을 화면에 출력한다(기본 동작).
 - `-ls`: 검색 결과를 긴 목록 형식으로 출력한다.
- **사용 예** `find ~ -name hello.c`
`find /tmp -user user10 -exec rm {} \;`

Directory & File

- 주요 명령어

- 파일 찾기

- 예: /usr/bin 디렉터리에서 ls 파일의 위치를 찾기

```
user1@myubuntu:~/linux_ex/ch2$ find /usr/bin -name ls
/usr/bin/ls
```

- 예: -user 옵션을 사용 하여 특정 사용자 계정이 소유자인 파일을 찾기

```
user1@myubuntu:~/linux_ex/ch2$ find /home -user user1
/home/user1
/home/user1/바탕화면
/home/user1/.bashrc
/home/user1/.cache
/home/user1/.cache/tracker3
/home/user1/.cache/tracker3/files
/home/user1/.cache/tracker3/files/http%3A%2F%2Ftracker.api.gnome.org%2Fontology%2Fv3%2F
tracker%23FileSystem.db-wal
/home/user1/.cache/tracker3/files/http%3A%2F%2Ftracker.api.gnome.org%2Fontology%2Fv3%2F
tracker%23Audio.db-shm
(생략)
```

Directory & File

- 주요 명령어

- 파일 찾기

- find 명령으로 검색한 모든 파일을 대상으로 동일한 작업을 수행하려면 -exec나 -ok 옵션 사용

```
user1@myubuntu:~/linux_ex/ch2$ find /tmp -user user1 -exec rm {} \;  
find: '/tmp/snap.firefox': 허가 거부  
find: '/tmp/systemd-private-2de14edc877647a39fc3b92a0d9fa6c8-power-profiles-daemon.service-B9S9zF': 허가 거부  
find: '/tmp/systemd-private-2de14edc877647a39fc3b92a0d9fa6c8-switcheroo-control.service-LYIMmg': 허가 거부  
find: '/tmp/systemd-private-2de14edc877647a39fc3b92a0d9fa6c8-systemd-timesyncd.service-t56bhl':  
허가 거부  
rm: 쓰기 보호된 일반 파일 '/tmp/.X1-lock'를(을) 지울까요? n  
rm: '/tmp/tracker-extract-3-files.1000'를 지울 수 없음: 디렉터리입니다  
find: '/tmp/systemd-private-2de14edc877647a39fc3b92a0d9fa6c8-colord.service-0ZtAdt': 허가 거부  
(생략)
```

Directory & File

- 주요 명령어

whereis

- **기능** 지정된 경로에서 명령의 바이너리 파일이나 매뉴얼 파일의 위치를 찾는다.
- **형식** whereis [옵션] [파일]
- **옵션** -b: 바이너리 파일만 검색한다.
-m: 매뉴얼 파일만 검색한다.
-s: 소스 파일만 검색한다.
- **사용 예** whereis ls

```
user1@myubuntu:~/linux_ex/ch2$ whereis mv
mv: /usr/bin/mv /usr/share/man/man1/mv.1.gz
```

Contents

- 리눅스 기초
- Directory and File
- 문서 편집
- Shell (셸)
- 접근 권한
- 프로세스 관리
- 리눅스 git 세팅

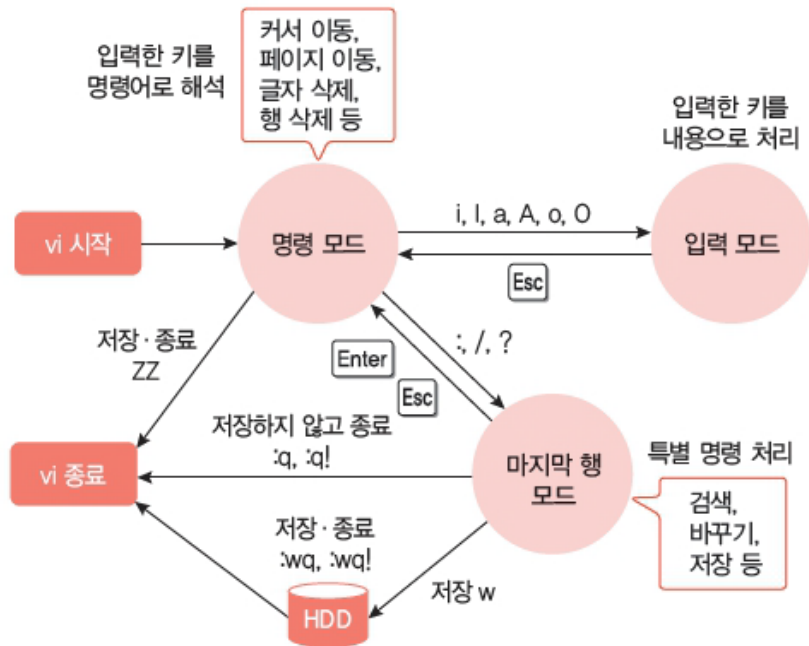
문서 편집

- vi
- nano
- gedit

구분	종류
행 단위 편집기	ed, ex, sed
화면 단위 편집기	vi, emacs(이맥스)
GUI 편집기	gedit

문서 편집

- vi
 - 가장 일반적으로 활용되는 기본 텍스트 에디터
 - 모드의 구분이 있음



```
[user1@localhost ~]$ vi test.txt
```

→ test.txt라는 파일이 열린다. 이 파일이 없으면 빈 파일이 열린다.

```
[user1@localhost ~]$ vi
```

→ 빈 파일이 열린다. 나중에 파일명을 정할 수 있다.

문서 편집

- 명령모드 -> 입력모드

i	현재 커서의 위치부터 입력(I)	I	현재 커서 줄의 맨 앞에서부터 입력(Shift + I)
a	현재 커서의 위치 다음 칸부터 입력(A)	A	현재 커서 줄의 맨 마지막부터 입력(Shift + A)

- 명령모드에서 커서 이동

h	커서를 왼쪽으로 한 칸 이동(H 와 같은 의미, H)	j	커서를 아래로 한 칸 이동(J 와 같은 의미, J)
k	커서를 위로 한 칸 이동(K 와 같은 의미, K)	l	커서를 오른쪽으로 한 칸 이동(L 와 같은 의미, L)

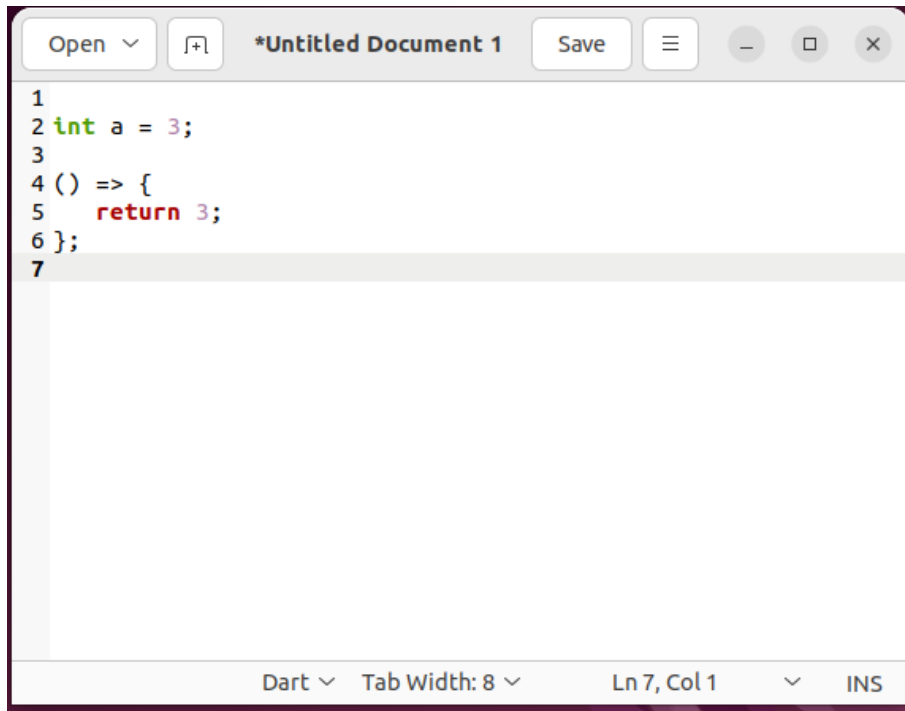
- 입력모드 -> 명령모드
 - ESC 키

문서 편집

- nano
 - 별도 모드가 없는 텍스트 에디터
 - nano -c 추천
 - 줄 정보 표시
 - 하단에 명령어 표기
- 기본 활용
 - 작성 후 Ctrl + X
 - 저장할 파일명 입력
 - 권한 문제 발생 시 sudo 로 실행

문서 편집

- gedit
 - GUI 기반 에디터
 - Windows 의 메모장과 동일
 - CLI 환경에서는 사용할 수 없음



Contents

- 리눅스 기초
- Directory and File
- 문서 편집
- Shell (셸)
- 접근 권한
- 프로세스 관리
- 리눅스 git 세팅

셸의 개념

- 리눅스의 셸
 - 명령과 프로그램을 실행할 때 사용하는 인터페이스
 - 사용자가 입력한 명령을 해석하여 커널에 전달하거나 커널의 처리 결과를 사용자에게 전달하는 역할
 - ubuntu에서 기본적으로 사용하는 셸은 bash(Bourne Again Shell)이며 배시셸이라고 읽음
- 셸의 기능
 - 명령어 해석기 기능
 - 프로그래밍 기능: 셸 스크립트
 - 사용자 환경 설정 기능

셸의 종류

- 본셸(sh): 최초의 셸, 유닉스 v7에서 처음 등장, 현재는 배시셸 등 다른 셸로 대체
- C셸(csh): 캘리포니아대학교에서 빌 조이가 개발, 2BSD 유닉스에서 발표
 - 앨리어스, 히스토리 기능 포함, 셸 스크립트 구문 형식이 C언어와 동일
- 콘셸(ksh): 1980년대 중반 AT&T 벨연구소에서 개발, SVR4 유닉스에서 발표
 - 본셸과 호환성 유지, 앨리어스, 히스토리 기능 제공
- 배시셸(bash): 1988년 브레인 폭스가 개발
 - 본셸과 호환성 유지, C셸/콘셸의 편리한 기능 모두 포함
 - 리눅스의 기본 셸로 제공
- 대시셸(dash): 1997년 허버스 슈가 리눅스에 이식
 - 본셸을 기반으로 개발, POSIX 표준 준수하며 작은 크기로 개발
 - 우분투는 6.10버전부터 본셸 대신 대시셸을 사용

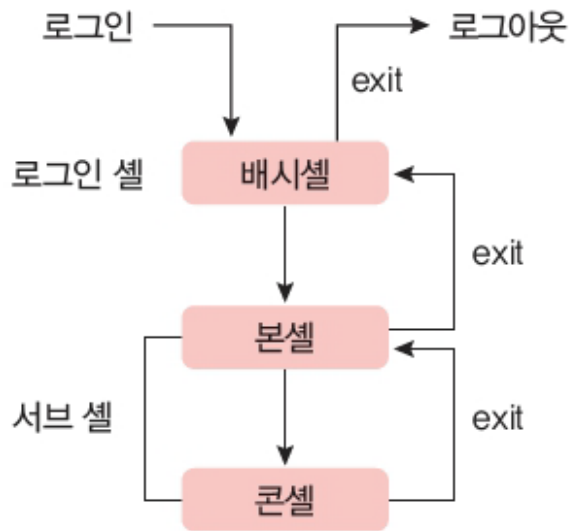
셸 기본 사용법

- 셸 확인

```
user1@myubuntu:~$ grep user1 /etc/passwd  
user1:x:1000:1000:user1,,,:/home/user1:/bin/bash
```

셀 기본 사용법

- 서브 셀 개념
 - 서브 셀: 사용자가 프롬프트에서 다른 셀을 실행하여 생성한 셀
 - 여러 개의 서브 셀을 사슬처럼 연결 가능
 - ctrl+d 또는 exit로 서브 셀 종료



셸 기본 사용법

- 셸 내장 명령
 - 셸 자체적으로 가지고 있는 명령 (e.g. cd)
 - /bin이나 /usr/bin 등 다른 디렉터리에 실행파일이 있음
 - file 명령으로 실행 파일 확인
 - 예를 들어 pwd 에 대한 명령어를 확인하고자 한다면

```
user1@myubuntu:~$ file /usr/bin/pwd
/usr/bin/pwd: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=401d991746e6ee4db275a4
fd4a860eab43145610, for GNU/Linux 3.2.0, stripped
```

- 결국 명령어들도 모두 파일의 일종

셸 기본 사용법

- 출력 명령

echo

- **기능** 화면에 한 줄의 문자열을 출력한다.
- **형식** echo [-n] [문자열]
- **옵션** -n: 마지막에 줄 바꿈을 하지 않는다.
- **사용 예**
echo
echo text
echo -n text

```
user1@myubuntu:~$ echo linux
linux
user1@myubuntu:~$ echo "linux ubuntu"
linux ubuntu
```

printf

- **기능** 자료를 형식화하여 화면에 출력한다.
- **형식** printf [형식] [인수]
- **옵션** 형식: %d, \n 등 C 언어의 printf 함수의 형식을 지정한다.
- **사용 예**
printf text
printf "text\n"
printf "%d\n" 100

```
user1@myubuntu:~$ printf linux
linuxuser1@myubuntu:~$ printf "linux ubuntu \n"
linux ubuntu
user1@myubuntu:~$ printf "%d + %d = %d\n" 10 10 20
10 + 10 = 20
```


셸 기본 사용법

- 특수 문자의 활용

- *

사용 예	의미
ls *	현재 디렉터리의 모든 파일과 서브 디렉터리를 나열한다. 서브 디렉터리의 내용도 출력한다.
cp */tmp	현재 디렉터리의 모든 파일을 /tmp 디렉터리 아래로 복사한다.
ls -F t*	t, tmp, temp와 같이 파일명이 t로 시작하는 모든 파일의 이름과 파일 종류를 출력한다. t도 해당한다는 데 주의한다.
cp *.txt ../ch3	확장자가 txt인 모든 파일을 상위 디렉터리 아래의 ch3 디렉터리로 복사한다.
ls -l h*d	파일명이 h로 시작하고 d로 끝나는 모든 파일의 상세 정보를 출력한다. hd, had, hard, h12345d 등 이 조건에 맞는 모든 파일의 정보를 볼 수 있다.

셸 기본 사용법

- 특수 문자의 활용
 - ? []

사용 예	의미
ls t?.txt	t 다음에 임의의 한 문자가 오고 파일의 확장자가 txt인 모든 파일의 이름을 출력한다. t1.txt, t2.txt, ta.txt 등이 해당된다. 단, t.txt는 제외한다.
ls -l tmp[135].txt	tmp 다음에 1, 3, 5 중 하나가 오고 파일의 확장자가 txt인 모든 파일의 이름을 출력한다. tmp1.txt, tmp3.txt, tmp5.txt 파일이 있으면 해당 파일의 상세 정보를 출력한다. 단, tmp.txt는 제외한다.
ls -l tmp[1-3].txt	[1-3]은 1부터 3까지의 범위를 의미한다. 따라서 ls -l tmp[123].txt와 결과가 같다. 즉 tmp1.txt, tmp2.txt, tmp3.txt 파일이 있으면 해당 파일의 상세 정보를 출력한다.
ls [0-9]*	파일명이 숫자로 시작하는 모든 파일의 목록을 출력한다.
ls [A-Za-z]*[0-9]	파일명이 영문자로 시작하고 숫자로 끝나는 모든 파일의 목록을 출력한다.

셸 기본 사용법

- 특수 문자의 활용

- ~ -

사용 예	의미
cp *.txt ~/ch3	확장자가 txt인 모든 파일을 현재 작업 중인 사용자의 홈 디렉터리 아래 tmp 디렉터리로 복사한다.
cp ~/user2/linux.txt .	user2라는 사용자의 홈 디렉터리 아래에서 linux.txt 파일을 찾아 현재 디렉터리로 복사한다.
cd -	이전 작업 디렉터리로 이동한다.

- ; |

사용 예	의미
date ; ls ; pwd	왼쪽부터 차례대로 명령을 실행한다. 즉 날짜를 출력한 후 현재 디렉터리의 파일 목록을 출력하고, 마지막으로 현재 작업 디렉터리의 절대 경로를 보여준다.
ls -al / more	루트 디렉터리에 있는 모든 파일의 상세 정보를 한 화면씩 출력한다. ls -al / 명령의 결과가 more 명령의 입력으로 전달되어 페이지 단위로 출력되는 것이다.

셸 기본 사용법

- 특수 문자의 활용

- \

사용 예	의미
ls -l t*	t*라는 이름을 가진 파일의 상세 정보를 출력한다. \ 없이 t*를 사용하면 t로 시작하는 모든 파일의 상세 정보를 출력한다.
echo \SHELL	\$SHELL을 화면에 출력한다. echo '\$SHELL'과 결과가 같다.

- > < >>

사용 예	의미
ls -l > res	ls -l 명령의 실행 결과를 화면이 아닌 res 파일에 저장한다.
ls -l >> res	ls -l의 명령의 결과를 res파일의 끝부분에 추가한다.
cat < text	cat 명령의 입력을 text 파일에서 받는다.

셸 기본 사용법

- History

- 사용자가 이전에 입력한 명령을 다시 불러내서 사용하는 것

history

- **기능** 히스토리(명령 입력 기록)를 출력한다.
- **형식** history

```
user1@myubuntu:/tmp$ history
```

(생략)

```
340 unalias cd
341 function cdpwd { cd $1;pwd; }
342 cdpwd /tmp
343 cdpwd
344 cdpwd /tmp
345 history
```

- 번호 지정 실행

```
user1@myubuntu:~/linux_ex/ch4$ !347
```

```
ls
```

```
out1 temp u.out
```

```
user1@myubuntu:~/linux_ex/ch4$ !l
```

```
ls
```

```
out1 temp u.out
```

→ 히스토리 번호로 재실행한다.

→ 명령의 앞 글자로 재실행한다.

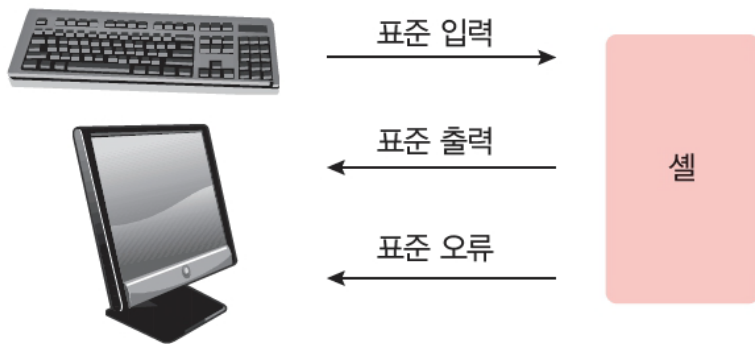
History

- 명령 편집 & 재실행

⬆, ⬇로 이전 명령을 불러온 다음 ⬅, ➡를 사용하여 명령의 잘못된 부분을 수정

입출력 방향 변경

- 표준 입출력 (standard input and output => stdio)



- 출력 리다이렉션
 - 명령의 결과를 화면 출력이 아닌 파일에 저장
 - 서버 로그 측면에서 중요
 - 파일 덮어쓰기 >

>

- 기능** 파일 리다이렉션(덮어쓰기)을 한다.
- 형식** 명령 1> 파일명
명령 > 파일명

입출력 방향 변경

- 출력 리다이렉션 예시

```
user1@myubuntu:~/linux_ex/ch4$ ls -al > out1
user1@myubuntu:~/linux_ex/ch4$ cat out1
합계 8
drwxrwxr-x 2 user1 user1 4096 12월 12 15:45 .
drwxrwxr-x 5 user1 user1 4096 12월 12 15:43 ..
-rw-rw-r-- 1 user1 user1    0 12월 12 15:45 out1
user1@myubuntu:~/linux_ex/ch4$ date > out1
user1@myubuntu:~/linux_ex/ch4$ cat out1
2021. 12. 12. (일) 15:45:45 KST
```

→ 명령의 결과를 out1 파일에 저장한다.

→ 파일 내용을 확인한다.

→ 명령의 결과를 out1 파일에 저장한다.

→ ls 명령의 실행 결과가 없어졌다.

입출력 방향 변경

- 파일에 내용 추가하기

>>

- **기능** 파일에 내용을 추가한다.
- **형식** 명령 >> 파일명

```
user1@myubuntu:~/linux_ex/ch4$ cat out1
```

```
Ubuntu Linux
```

```
I love Linux.
```

```
user1@myubuntu:~/linux_ex/ch4$ date >> out1
```

```
user1@myubuntu:~/linux_ex/ch4$ cat out1
```

```
Ubuntu Linux
```

```
I love Linux.
```

```
2021. 12. 12. (일) 15:50:18 KST
```

→ 기존 파일 내용을 확인한다.

→ 리다이렉션한다(내용 추가).

→ 파일 내용을 확인한다.

→ 추가된 내용

입출력 방향 변경

- 오류는 화면 출력이 기본

```
user1@myubuntu:~/linux_ex/ch4$ ls
```

```
out1
```

→ 정상 실행(표준 출력)

```
user1@myubuntu:~/linux_ex/ch4$ ls /abc
```

```
ls: '/abc'에 접근할 수 없습니다: 그런 파일이나 디렉터리가 없습니다
```

→ 오류 메시지(표준 오류)

```
user1@myubuntu:~/linux_ex/ch4$ ls > ls.out
```

→ 표준 출력 리다이렉션

```
user1@myubuntu:~/linux_ex/ch4$ ls /abc > ls.err
```

→ 표준 출력 리다이렉션

```
ls: '/abc'에 접근할 수 없습니다: 그런 파일이나 디렉터리가 없습니다
```

→ 오류 메시지가 화면에 출력된다.

```
user1@myubuntu:~/linux_ex/ch4$ cat ls.err
```

→ 오류 메시지가 저장되지 않았다.

```
user1@myubuntu:~/linux_ex/ch4$ cat ls.out
```

→ 표준 출력 내용이 출력된다.

```
ls.out
```

```
out1
```

입출력 방향 변경

- 따라서, 오류 메시지를 파일로 저장하려면 별도의 처리가 필요함

2>

- 기능** 표준 오류 메시지를 파일에 저장한다.
- 형식** 명령 2> 파일명

```
user1@myubuntu:~/linux_ex/ch4$ ls /abc 2> ls.err
```

→ 표준 오류를 리다이렉션한다.

```
user1@myubuntu:~/linux_ex/ch4$ cat ls.err
```

```
ls: '/abc'에 접근할 수 없습니다: 그런 파일이나 디렉터리가 없습니다
```

→ 파일에 저장된 메시지이다.

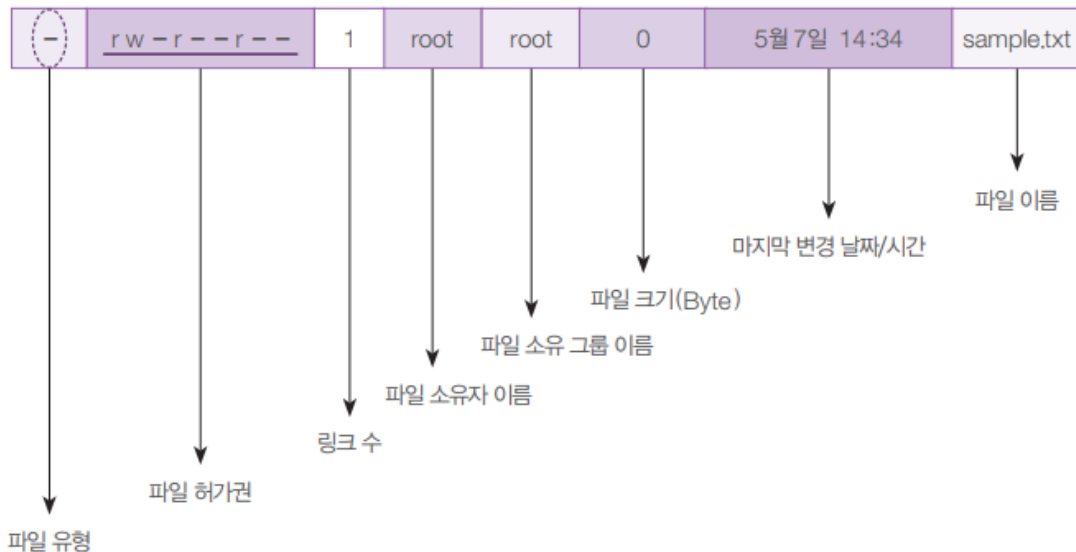
Contents

- 리눅스 기초
- Directory and File
- 문서 편집
- Shell (셸)
- 접근 권한
- 프로세스 관리
- 리눅스 git 세팅

파일의 속성

- 파일 속성 정보 보기

```
root@server: ~/바탕화면
root@server:~/바탕화면# touch sample.txt
root@server:~/바탕화면# ls -l sample.txt
-rw-r--r-- 1 root root 0 5월 7 14:34 sample.txt
root@server:~/바탕화면#
```



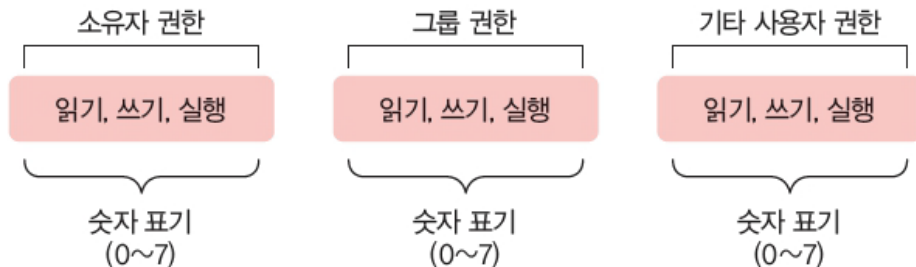
파일의 속성

- 파일 유형
 - 디렉터리일 경우에는 d, 일반적인 파일일 경우에는 -가 표시
- 파일 허가권(Permission)
 - "rw-", "r--", "r--" 3개씩 끊어서 읽음 (r은 read, w는 write, x는 execute의 약자)
 - 첫 번째 "rw-"는 소유자(User)의 파일접근 권한
 - 두 번째의 "r--"는 그룹(Group)의 파일접근 권한
 - 세 번째의 "r--"는 그 외의 사용자(Other)의 파일접근 권한
 - 숫자로도 표시 가능 (8진수)

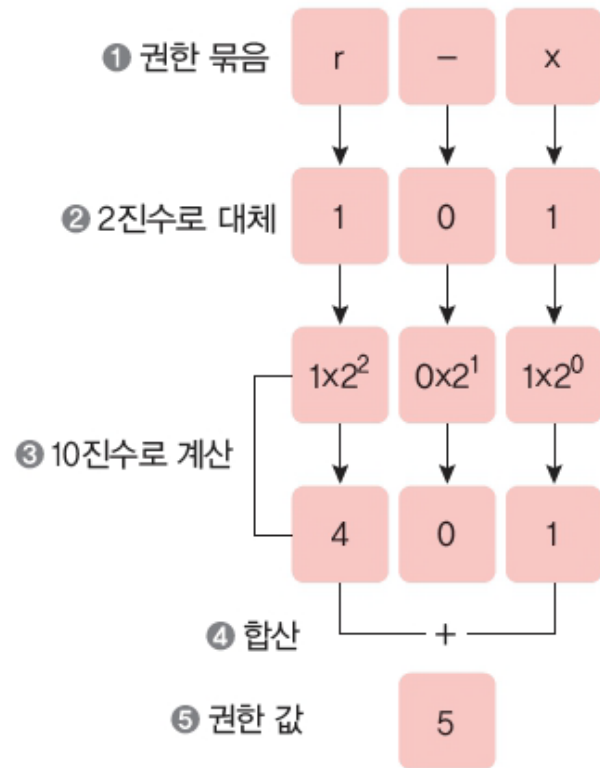
소유자(User)			그룹(Group)			그 외 사용자(Other)		
r	w	-	r	-	-	r	-	-
4	2	0	4	0	0	4	0	0
6			4			4		

파일의 속성

- 숫자로 환산하는 방법



접근 권한	숫자 모드	접근 권한	숫자 모드
rwXrwxrwx	777	rw-r--r--	644
rwXr-xr-x	755	rwX-----	700
rw-rw-rw	666	rw-r-----	640
r-xr-xr-x	555	r-----	400



파일의 소유와 허가권

- chmod 명령
 - 파일 허가권 변경 명령어
 - 예) `$ chmod 777 sample.txt`

chmod

- **기능** 파일이나 디렉터리의 접근 권한을 변경한다.
- **형식** `chmod [옵션] 권한 파일(디렉터리)`
- **옵션** `-R`: 하위 디렉터리까지 모두 변경할 수 있다.

Contents

- 리눅스 기초
- Directory and File
- 문서 편집
- Shell (셸)
- 접근 권한
- 프로세스 관리
- 리눅스 git 세팅

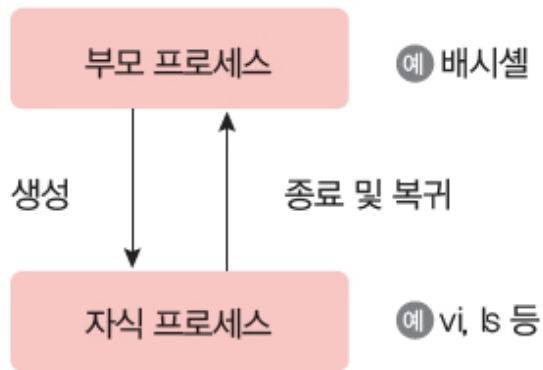
프로세스의 개념

- 프로세스

- 현재 시스템에서 실행 중인 프로그램 (메모리에 할당되어있는 상태)
- 리눅스는 다중 프로세스 시스템으로 동시에 여러 프로세스 실행
- 프로세스를 자유자재로 전환하거나, 끌 수 있어야 함 = 윈도우에서 다른 창으로 전환하거나 끌 수 있어야 함

- 프로세스의 부모-자식 관계

- 리눅스에서 모든 프로세스는 부모-자식 관계가 있음
- 부모프로세스가 자식프로세스 생성



프로세스의 개념

- 프로세스 번호
 - 각 프로세스는 고유한 번호를 가짐: PID
 - 1번 프로세스는 system, 2번 프로세스는 kthreadd
- 프로세스의 종류
 - Foreground 프로세스
 - Background 프로세스
 - 데몬 프로세스: 특정 서비스를 제공하기 위해 존재하며 리눅스 커널에 의해 실행되며, 사용자와 상호작용하지 않고 자동으로 관리됨
 - 고아 프로세스: 자식 프로세스가 아직 실행 중인데 부모 프로세스가 먼저 종료되면 자식 프로세스는 고아 프로세스가 됨
 - 좀비 프로세스
 - 자식 프로세스가 실행을 종료했는데도 프로세스 테이블 목록에 남아 있는 경우
 - 좀비 프로세스가 증가하면 프로세스 테이블의 용량이 부족해서 정상적인 프로세스가 실행되지 않을 수도 있음

프로세스 관리 명령

- 주요 명령어

ps

- 기능** 현재 실행 중인 프로세스에 대한 정보를 출력한다.
- 형식** ps [옵션]
- 옵션** <유닉스 옵션>
 - e: 시스템에서 실행 중인 모든 프로세스의 정보를 출력한다.
 - f: 프로세스에 대한 자세한 정보를 출력한다.
 - u uid: 특정 사용자에게 대한 모든 프로세스의 정보를 출력한다.
 - p pid: pid로 지정한 특정 프로세스의 정보를 출력한다.<BSD 옵션>
 - a: 터미널에서 실행시킨 프로세스의 정보를 출력한다.
 - u: 프로세스 소유자 이름, CPU 사용량, 메모리 사용량 등 상세 정보를 출력한다.
 - x: 시스템에서 실행 중인 모든 프로세스의 정보를 출력한다.<GNU 옵션>
 - pid PID 목록: 목록으로 지정한 특정 PID 정보를 출력한다.
- 사용 예**
 - ps
 - ps -ef
 - ps aux

프로세스 관리 명령

- ps 명령의 옵션 유형
 - 유닉스(SVR4) 옵션: 묶어서 사용할 수 있고, -으로 시작한다(예: -ef).
 - BSD 옵션: 묶어서 사용할 수 있고, -으로 시작하지 않는다(예 aux).
- 프로세스 상세 정보 출력하기: -f 옵션
 - PPID와 C(CPU 사용량), 시작 시간 등의 정보가 추가로 출력

항목	의미	항목	의미
UID	프로세스를 실행한 사용자 ID	STIME	프로세스의 시작 날짜나 시간
PID	프로세스 번호	TTY	프로세스가 실행된 터미널의 종류와 번호
PPID	부모 프로세스 번호	TIME	프로세스 실행 시간
C	CPU 사용량(% 값)	CMD	실행되고 있는 프로그램 이름(명령)

프로세스 관리 명령

- 전체 프로세스 목록 출력하기(유닉스 옵션): - e 옵션과 - f 옵션
 - e 옵션은 시스템에서 실행 중인 모든 프로세스를 출력
 - 전체 프로세스의 더 자세한 정보를 확인하려면 -e 옵션과 -f 옵션을 함께 사용

```
user1@myubuntu:~$ ps -ef | more
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	12월18	?	00:00:01	/sbin/init splash
root	2	0	0	12월18	?	00:00:00	[kthreadd]
root	3	2	0	12월18	?	00:00:00	[rcu_gp]
root	4	2	0	12월18	?	00:00:00	[rcu_par_gp]
root	6	2	0	12월18	?	00:00:00	[kworker/0:0H-events_highpri]

(생략)

프로세스 관리 명령

- 터미널에서 실행시킨 프로세스 정보 출력하기: a 옵션

```
user1@myubuntu:~$ ps a
  PID TTY          STAT TIME  COMMAND
  1586 tty2      Ssl+   0:00 /usr/libexec/gdm-wayland-session env GNOME_SHELL_SESSION_
MODE=ubuntu /usr/bin/gnome-session --sessio
  1589 tty2      Sl+    0:00 /usr/libexec/gnome-session-binary --systemd --session
=ubuntu
  2183 pts/0      Ss     0:00  bash
  2951 pts/0      R+     0:00  ps a
```

문자	의미	비고
R	실행 중(running)	
S	인터럽트가 가능한 대기(sleep) 상태	
T	작업 제어에 의해 정지된(stopped) 상태	
Z	좀비 프로세스(defunct)	
STIME	프로세스의 시작 날짜나 시간	
s	세션 리더 프로세스	BSD 형식
+	포그라운드 프로세스 그룹	
l(소문자 l)	멀티스레드	

프로세스 관리 명령

- 터미널에서 실행시킨 프로세스 정보 출력하기: a 옵션, u 옵션

```
user1@myubuntu:~$ ps au
```

```
USER      PID  %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
user1    1586  0.0  0.1 172388  6036 tty2      Ssl+ 10:12   0:00 /usr/libexec/gdm-
wayland-session env GNOME_SHELL_SESSION_MODE=
user1    1589  0.0  0.3 231444 15432 tty2      Sl+   10:12   0:00 /usr/libexec/
gnome-session-binary --systemd --session=ubuntu
user1    2183  0.0  0.1  21036  5044 pts/0    Ss   10:12   0:00 bash
user1    2957  0.0  0.0  22468  3492 pts/0    R+   11:04   0:00 ps au
```

항목	의미
USER	사용자 계정 이름
%CPU	퍼센트로 표시한 CPU 사용량
%MEM	퍼센트로 표시한 물리적 메모리 사용량
VSZ	사용 중인 가상 메모리의 크기(KB)
RSS	사용 중인 물리적 메모리의 크기(KB)
START	프로세스 시작 시간

프로세스 관리 명령

- 전체 프로세스 목록 출력하기(BSD 옵션): aux 옵션
 - aux 옵션은 -ef 옵션처럼 시스템에서 실행 중인 모든 프로세스에 대한 자세한 정보를 출력

```
user1@myubuntu:~$ ps aux | more
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.0	0.2	164520	10692	?	Ss	12월18	0:01	/sbin/init splash
root	2	0.0	0.0	0	0	?	S	12월18	0:00	[kthreadd]
root	3	0.0	0.0	0	0	?	I<	12월18	0:00	[rcu_gp]
root	4	0.0	0.0	0	0	?	I<	12월18	0:00	[rcu_par_gp]

프로세스 관리 명령

- 특정 프로세스 정보 검색하기: ps와 grep 활용
 - 'ps -ef | grep 명령' 의 형태로 연결해서 사용

```
user1@myubuntu:~$ ps -ef | grep bash
user1      2096      2095  0 13:18 pts/0    00:00:00 -bash
user1      2111      2096  0 13:18 pts/0    00:00:00 grep --color=auto bash
```

- ps -ef 와 ps aux 중에서 어떤 것이 더 좋나요?
 - 부모-자식 프로세스 관계를 볼 때 : -ef
 - 모든 user 의 프로세스 상태를 볼 때 : aux

프로세스 관리 명령

- 시그널

- 무언가 발생했음을 프로세스에게 알리는 간단한 메시지
- 무엇이 발생했는지를 나타내는, 미리 정의된 상수를 사용
- 시그널은 번호로 구분되며 이름을 가지고 있음
- 시그널을 받은 프로세스는 기본적으로 종료

```
user1@myubuntu:~$ kill -l
```

1) SIGHUP	2) SIGINT	3) SIGQUIT	4) SIGILL	5) SIGTRAP
6) SIGABRT	7) SIGBUS	8) SIGFPE	9) SIGKILL	10) SIGUSR1
11) SIGSEGV	12) SIGUSR2	13) SIGPIPE	14) SIGALRM	15) SIGTERM
16) SIGSTKFLT	17) SIGCHLD	18) SIGCONT	19) SIGSTOP	20) SIGTSTP

시그널	번호	기본 처리	의미
SIGHUP	1	종료	터미널과 연결이 끊겼을 때 발생한다.
SIGINT	2	종료	인터럽트로 사용자가 Ctrl+C 를 입력하면 발생한다.
SIGQUIT	3	종료, 코어덤프	종료 신호로 사용자가 Ctrl+Q 를 입력하면 발생한다.
SIGKILL	9	종료	이 시그널을 받은 프로세스는 무시할 수 없으며 강제로 종료된다.
SIGALRM	14	종료	알람에 의해 발생한다.
SIGTERM	15	종료	kill 명령이 보내는 기본 시그널이다.

프로세스 관리 명령

- 프로세스 종료하기

kill

- 기능** 지정한 시그널을 프로세스에게 보낸다.
- 형식** kill [-시그널] PID...
- 시그널** 2: 인터럽트 시그널을 보낸다(**Ctrl**+c).
9: 프로세스를 강제로 종료한다.
15: 프로세스와 관련된 파일들을 정리하고 종료한다. 종료되지 않는 프로세스가 있을 수 있다.
- 사용 예** kill 1001
kill -15 1001
kill -9 1001

프로세스 관리 명령

- 프로세스 종료 예시

- 1번 터미널

```
user1@myubuntu:~$ man ps
PS(1)

NAME
      ps - report a snapshot c
(생략)
```

```
Manual page ps(1) line 1 (press h for help or q to quit)종료됨
user1@myubuntu:~$
```

2번 터미널

```
user1@myubuntu:~$ ps -fp $(pgrep -x man)
UID          PID    PPID  C  STIME TTY          TIME CMD
user1        2290    2172  0  14:48 pts/1      00:00:00 man ps

user1@myubuntu:~$ kill 2290
```

작업 제어

- 작업 제어 (Service control)

- 한 터미널에서 동시에 여러 프로세스를 실행하고 관리
- C셸에서 처음 개발되었고 4.1BSD 커널에 포함
- 대부분의 유닉스 셸과 리눅스 셸에 포함하여 제공
- 작업 제어 도구가 관리하는 프로세스를 작업이라고 하며, 전환, 일시중지, 종료 등이 가능함

- Foreground service

- 사용자가 입력한 명령이 실행되어 결과가 출력될 때까지 기다리는 방식으로 처리되는 프로세스를 포그라운드 프로세스

```
user1@myubuntu:~$ sleep 100
```

→ 포그라운드 작업

→ sleep 명령이 끝날 때까지 기다려야 한다.

작업 제어

- Background service

- 백그라운드 방식으로 명령을 실행하면 명령의 처리가 끝나는 것과 관계없이 곧바로 프롬프트가 출력되어 사용자가 다른 작업을 계속할 수 있음
- 한 터미널에서 여러 개의 프로세스를 동시에 실행 가능
- 명령을 백그라운드로 실행하려면 명령의 마지막에 &(앰퍼샌드) 기호를 추가
- 백그라운드로 작업을 실행하면 프롬프트가 바로 나옴

```
user1@myubuntu:~$ sleep 100 & → 백그라운드 작업
```

```
user1@myubuntu:~$ → 프롬프트가 바로 나와 다른 명령을 실행시킬 수 있다.
```

작업 제어

- 작업 목록 보기

jobs

- 기능** 백그라운드 작업을 모두 보여준다. 특정 작업 번호를 지정하면 해당 작업의 정보만 보여준다.
- 형식** jobs [%작업 번호]
- %작업번호** %번호: 해당 번호의 작업 정보를 출력한다.
%+ 또는 %%: 작업 순서가 +인 작업 정보를 출력한다.
%-: 작업 순서가 -인 작업 정보를 출력한다.
- 사용 예** jobs %1
jobs

작업 제어

- 작업 목록 보기
 - jobs 명령 실행 예시

```
user1@myubuntu:~$ jobs
```

```
[1]-  실행중                sleep 100 &
```

```
[2]+  실행중                find / -name passwd > pw.dat 2>&1 &
```

항목	출력 예	의미
작업 번호	[1]	작업 번호로서 백그라운드로 실행할 때마다 순차적으로 증가한다([1], [2], [3],...).
작업 순서	+	작업 순서를 표시한다. +: 가장 최근에 접근한 작업 -: + 작업 바로 전에 접근한 작업 - 공백: 그 외의 작업
상태	실행중	작업 상태를 표시한다. - 실행중: 현재 실행되고 있다. - 완료: 작업이 정상적으로 종료되었다. - 종료됨: 작업이 비정상적으로 종료되었다. - 멈춤: 작업이 잠시 중단되었다.
명령	sleep 100 &	백그라운드로 실행 중인 명령이다.

작업 제어

- 작업 전환하기

명령	기능
<code>Ctrl+z</code> 또는 <code>stop %작업 번호</code>	포그라운드 작업을 중지한다(종료하는 것이 아니라 잠시 중단하는 것이다).
<code>bg %작업 번호</code>	작업 번호가 지시하는 작업을 백그라운드 작업으로 전환한다.
<code>fg %작업 번호</code>	작업 번호가 지시하는 작업을 포그라운드 작업으로 전환한다.

- 예시

```
user1@myubuntu:~$ jobs
user1@myubuntu:~$ sleep 100
^Z
[1]+  멈춤                  sleep 100
user1@myubuntu:~$ bg %1
[1]+  sleep 100 &
user1@myubuntu:~$ jobs
[1]+  실행중                sleep 100 &
user1@myubuntu:~$ fg
sleep 100
```

→ 백그라운드 작업이 없다.
→ 포그라운드로 실행한다.
→ `Ctrl+z`로 일시 중지한다.
→ 일시 중지된 상태다.
→ 백그라운드로 전환한다.
→ 백그라운드로 실행 중이다.
→ 포그라운드로 전환한다.
→ 포그라운드로 실행 중이다.

작업 제어

- 작업 종료하기
 - 포그라운드 작업은 ctrl+c를 입력하면 대부분 종료
 - 또는 PID를 검색하여 kill 명령으로 강제 종료

```
user1@myubuntu:~$ sleep 100
```

→ 포그라운드로 실행 중이다.

```
^C
```

→ 강제 종료한다.

```
user1@myubuntu:~$
```

- 백그라운드 작업은 kill 명령으로 강제 종료

```
user1@myubuntu:~$ sleep 100&
```

→ 백그라운드로 실행 중이다.

```
[1] 53864
```

```
user1@myubuntu:~$ kill %1
```

→ 강제 종료한다.

```
user1@myubuntu:~$
```

```
[1]+  종료됨
```

```
sleep 100
```

→ 종료 메시지가 출력된다.

작업 제어

- 로그아웃 후에도 백그라운드 작업 계속 실행하기
 - 백그라운드 작업을 실행한 터미널이 종료되거나 사용자가 로그아웃하면 실행 중이던 백그라운드 작업도 함께 종료
 - 로그아웃한 다음에도 작업이 완료될 때까지 백그라운드작업을 실행해야 할 경우에 nohup 명령을 사용
 - 실행의 결과는 nohup.out 파일에 저장 (파일명 변경 가능)

nohup

- **기능** 로그아웃한 후에도 백그라운드 작업을 계속 실행한다.
- **형식** nohup 명령&

작업 제어

- nohup 예시

```
user1@myubuntu:~$ nohup find / -name passwd &  
[1] 53629  
user1@myubuntu:~$ nohup: 입력 무시 및 'nohup.out' 에 출력 추가
```



```
user1@myubuntu:~$ more nohup.out  
/usr/bin/passwd  
/usr/share/doc/passwd  
/usr/share/bash-completion/completions/passwd  
/usr/share/lintian/overrides/passwd  
find: '/lost+found': 허가 거부  
find: '/run/gdm3': 허가 거부  
find: '/run/udisks2': 허가 거부  
find: '/run/cups/certs': 허가 거부  
(생략)
```

```
user1@myubuntu:~$ nohup find / -name passwd > pw.dat 2>&1 &  
[1] 53878  
user1@myubuntu:~$ exit
```



```
user1@myubuntu:~$ more pw.dat  
nohup: 입력 무시  
/usr/bin/passwd  
/usr/share/doc/passwd  
/usr/share/bash-completion/completions/passwd  
/usr/share/lintian/overrides/passwd  
find: '/lost+found': 허가 거부  
find: '/run/gdm3': 허가 거부  
find: '/run/udisks2': 허가 거부  
find: '/run/cups/certs': 허가 거부  
(생략)
```

작업 예약

- 정해진 시간에 한 번 실행

at

- 기능** 예약한 명령을 정해진 시간에 실행한다.
- 형식** at [옵션] [시간]
- 옵션**
 - l: 현재 실행 대기 중인 명령의 전체 목록을 출력한다(atq 명령과 동일).
 - r 작업 번호: 현재 실행 대기 중인 명령 중 해당 작업 번호를 삭제한다(atrm과 동일).
 - m: 출력 결과가 없더라도 작업이 완료되면 사용자에게 메일로 알려준다.
 - f 파일: 표준 입력 대신 실행할 명령을 파일로 지정한다.
- 사용 예**
 - at -m 0730 tomorrow
 - at 10:00 pm
 - at 8:15 am May 30

작업 예약

- 정해진 시간에 한 번 실행 : at

- at 명령어 설치

```
user1@myubuntu:~$ sudo apt install at
```

- 명령 설정

```
user1@myubuntu:~/linux_ex/ch6$ at 04:30 pm
warning: commands will be executed using /bin/sh
at> /usr/bin/ls -l ~user1 > ~user1/at.out
at> <EOT>
job 2 at Sun Dec 19 16:50:00 2021
```

→ 시간을 지정한다.

→ 실행할 명령을 지정한다.

→ **Ctrl**+d 입력으로 종료한다.

→ 작업 예약을 완료한다.

- 시간 설정 방식

- at 4pm + 3 days: 지금부터 3일 후 오후 4시에 작업을 수행
 - at 10am Jul 31: 7월 31일 오전 10시에 작업을 수행
 - at 1am tomorrow: 내일 오전 1시에 작업을 수행
 - at 10:00am today: 오늘 오전 10시에 작업을 수행

작업 예약

- at 작업 목록 확인하기

- -l

```
user1@myubuntu:~/linux_ex/ch6$ at -l
2          Sun Dec 19 16:50:00 2021 a user1
```

- atq 명령

atq

- **기능** 현재 사용자의 등록된 작업 목록을 보여준다. 슈퍼유저일 경우 모든 사용자의 작업 목록을 보여준다.
- **형식** atq

```
user1@myubuntu:~/linux_ex/ch6$ atq
2          Sun Dec 19 16:50:00 2021 a user1
```


작업 예약

- at 작업 삭제하기
 - -d 옵션
 - atrm 명령

atrm

- **기능** 지정된 작업 번호의 작업을 삭제한다.
- **형식** atrm 작업 번호

```
user1@myubuntu:~/linux_ex/ch6$ atq
3      Mon Dec 20 01:00:00 2021 a user1
4      Sun Dec 19 22:00:00 2021 a user1
2      Sun Dec 19 16:50:00 2021 a user1
```



```
user1@myubuntu:~/linux_ex/ch6$ at -d 3
user1@myubuntu:~/linux_ex/ch6$ atrm 4
user1@myubuntu:~/linux_ex/ch6$ atrm 2
user1@myubuntu:~/linux_ex/ch6$ atq
user1@myubuntu:~/linux_ex/ch6$
```

작업 예약

- at 작업 파일 확인하기
 - at로 생성된 작업 파일은 /var/spool/cron/atjobs 디렉터리에 저장
 - 작업 번호로 파일이 생성
 - atjobs 디렉터리의 내용은 sudo 명령을 사용해야 함

```
user1@myubuntu:~/linux_ex/ch6$ ls -l /var/spool/cron/atjobs
ls: '/var/spool/cron/atjobs' 디렉터리를 열 수 없음: 허가 거부
user1@myubuntu:~/linux_ex/ch6$ sudo ls -l /var/spool/cron/atjobs
합계 4
-rwx----- 1 user1 daemon 2982 12월 19 16:38 a0000201a10c56
```

작업 예약

- 정해진 시간에 반복 실행

crontab

- **기능** 사용자의 crontab 파일을 관리한다.
- **형식** crontab [-u 사용자ID] [옵션] [파일명]
- **옵션** -e: 사용자의 crontab 파일을 편집한다.
-l: crontab 파일의 목록을 출력한다.
-r: crontab 파일을 삭제한다.
- **사용 예**
crontab -l
crontab -u user1 -e
crontab -r

작업 예약

- crontab 파일 생성 및 편집
 - `$ crontab -e`
 - vi, nano 등의 편집기로 입력 후 저장
- crontab 파일 내용 확인
 - `$ crontab -l`
- crontab 삭제
 - `$ crontab -r`
 - 특정 사용자의 crontab 삭제
 - `$ crontab -u user1 -r`

작업 예약

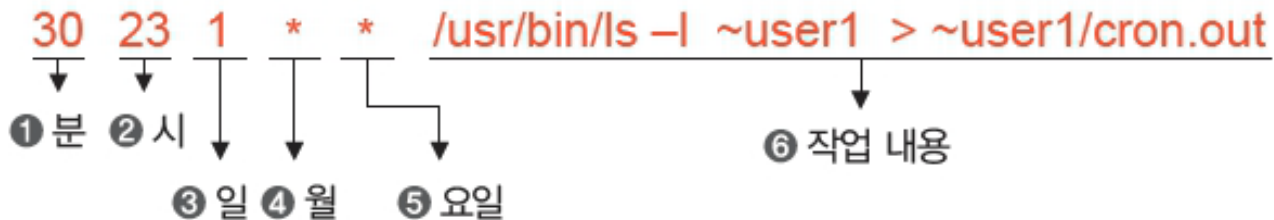
- crontab 파일 형식
 - crontab 명령으로 관리하는 파일은 사용자별로 생성
 - crontab 파일에는 여러 개의 작업을 저장할 수 있으며 한 행에 하나의 작업을 설정
 - crontab 파일의 한 행은 다음과 같이 여섯 항목으로 구성

분(0~59)	시(0~23)	일(1~31)	월(1~12)	요일(0~6)	작업 내용
---------	---------	---------	---------	---------	-------

- 요일은 0이 일요일, 1이 월요일, 6이 토요일을 의미
- 각 항목은 공백문자로 구분
- 항목의 값이 *이면 해당 항목의 모든 값을 의미

작업 예약

- crontab 항목 설정 예



- ④번의 월 항목은 매월을 의미, ⑤번의 요일은 모든 요일을 의미
- 따라서 이 명령은 매월 1일 23시 30분에 지정한 작업을 실행하라는 뜻
- `* / 5 * * * *` (명령어)
 - 매 5분마다
- `11,22,33 * * * *` (명령어)
 - 매 11, 22, 33분마다
- `0 1-6 * * *` (명령어)
 - 매일 오전 1-6시 사이 정각마다 실행
- `* 9 * * 1-5` (명령어)
 - 평일 오전 9시부터 10시 사이 매 분마다 실행

작업 예약

- 재부팅 시의 작업 예약
 - crontab 에 다음처럼 입력
 - @reboot 실행시키고자 하는 명령어
 - 예시
 - @reboot touch ~/a.txt
 - 재부팅 시 a.txt 파일에 접근하여 접근 시간이 변경됨

작업 예약

- Crontab 의 유용한 활용 사례

- 웹 서버 배포 등 지속적으로 동작해야하는 프로세스를 재부팅 후 자동으로 실행시킬 때
 - Learner Lab 환경에서 4시간마다 끊어지는 작업을 자동으로 실행시킬 때에도 활용
- 정해진 시간에 정보를 크롤링해야하는 작업 시
- 반복되는 작업을 편리하게 자동화할 수 있어서 시간과 노력을 크게 절감 가능

Contents

- 리눅스 기초
- Directory and File
- 문서 편집
- Shell (셸)
- 접근 권한
- 프로세스 관리
- [리눅스 git 세팅](#)

git version 확인

- ubuntu 22.04 버전에는 git 이 기본적으로 설치되어 있음
- \$ git --version

```
ubuntu@ip-172-31-30-124:~$ git --version  
git version 2.34.1
```

private repository clone

- (당연히) 권한 문제로 clone 이 불가능함
 - github 입장에서 AWS 컴퓨터는 새로운 또하나의 컴퓨터일 뿐

HTTPS 의 경우

```
ubuntu@ip-172-31-30-124:~$ git clone https://github.com/kw-ic-oss/test2.git
Cloning into 'test2'...
Username for 'https://github.com': kdpark-phd
Password for 'https://kdpark-phd@github.com':
remote: Support for password authentication was removed on August 13, 2021.
remote: Please see https://docs.github.com/get-started/getting-started-with-git/about-remote-repositories#cloning-with-https-urls for information on currently recommended modes of authentication.
fatal: Authentication failed for 'https://github.com/kw-ic-oss/test2.git/'
```

SSH 의 경우

```
ubuntu@ip-172-31-30-124:~$ git clone git@github.com:kw-ic-oss/test2.git
Cloning into 'test2'...
The authenticity of host 'github.com (140.82.112.4)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDA0zPMSvHdkr4UvCOqU.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
git@github.com: Permission denied (publickey).
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
```

private repository clone

- HTTPS 의 경우
 - personal access token 을 활용해야함
 - <https://docs.github.com/ko/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>
 - 계정 비밀번호 외, 새로운 접속용 토큰을 발행함
- SSH 의 경우
 - ssh 를 등록하여야 함
 - 직전 수업자료 p.53 부터 참고
 - ssh-keygen 활용하여 키 발급하고, 해당 키파일 (.pub) 내용을 github 의 ssh 키로 등록함
 - 신뢰할 수 있는 개인용 컴퓨터 및 서버에서 활용

Now, you should be able to

- local 컴퓨터에서 작성한 코드를 서버로 옮기기 (단, github 를 활용하여)
 1. github repo 를 local 컴퓨터로 clone
 2. local 컴퓨터에서 코드 수정
 3. github repo 에 push
 4. 서버 컴퓨터에서 git clone
- 이러한 방식은 개발 환경과 배포 환경의 코드가 동일하게 유지되는데 있어서 중요한 역할을 함
 - CI/CD 의 가장 기초가 됨
 - Continuous Integration / Continuous Deployment

간단한 html 파일을 배포하기

- python 의 http.server 를 활용하여 배포

- `$ sudo python3 -m http.server 80 --directory [작업디렉토리]`

```
ubuntu@ip-172-31-30-124:~$ sudo python3 -m http.server 80 --directory ~/sample
Serving HTTP on 0.0.0.0 port 80 (http://0.0.0.0:80/) ...
```

- 현재 shell 이 위치한 폴더에서 배포하고자 한다면 --directory 는 생략 가능
- public IP 주소로 접속하여 확인
- 터미널 종료 시, 작업이 끊어짐
- 이 프로세스를 백그라운드에서 동작시키려면?
- EC2 가 재부팅될 때, 자동으로 실행시키려면?

Thank you

Q & A